

Monthly outcome: where the robot is

M15 • 19 Dec 2027 - 20 Jan 2028 • 75 hours

Compare motion-based position estimation, localisation on a known map and joint reconstruction of a map and trajectory. Deliver a reproducible setup for 2D and visual SLAM, a trajectory-error table and an analysis of conditions that make estimates unreliable. This introduces global navigation for the future walking robot; a complete global-navigation implementation does not become a prerequisite for stable walking.

Connecting the body, legs and camera

The robot moves its body, measures motion with an IMU, observes surfaces with a camera and sometimes knows its support positions. Wheel odometry uses wheel geometry; leg odometry uses kinematics and an assumption of stationary contact; visual odometry uses an image sequence; inertial odometry integrates angular velocity and acceleration. For each source, record an assumption that can fail: slip, loss of contact, lack of texture or sensor bias.

What to bring from earlier modules

You need transformation matrices, a basic EKF, ROS 2 tf/rosbag, RGB-D loading and the M14 understanding of ICP. Check the yaw sign, quaternion component order, time units and camera-to-body transform. If these checks fail, first correct them using three known positions and orientations. Do not try to compensate for a coordinate-frame error by tuning filter noise.

Minimum experimental setup

The main 2D experiment uses a short simulator recording containing LaserScan, odometry, IMU and exact ground truth. Use one route: a straight section, a turn, a return to the start and a repetitive corridor. For the visual method, use an RGB-D sequence from the same simulator or a TUM recording with a reference trajectory. If the scenes differ, compare failure causes and normalised measures. Incomparable ATE values do not establish that one sensor is more accurate than another.

Continue in Ubuntu 24.04 and ROS 2 Jazzy. Use compatible versions of slam_toolbox and the chosen visual method; save versions and parameters. You do not need to write a second complete SLAM system from scratch: integrate an existing baseline or build a small pose graph from RGB-D odometry and verified loop closures. Save a map, trajectory and quantitative evaluation; an RViz visualisation alone is insufficient.

Organising 75 hours and comparing methods

Time allocation

1-20: odometry, coordinate frames, EKF and error growth; 10 hours of analysis and 10 of practice. 21-40: occupancy grids, inverse sensor models, scan matching and AMCL; 10+10. 41-60: pose graphs, nonlinear least squares, covariance and loop closure; 10+10. 61-75: ATE/RPE, degraded data, LiDAR-versus-vision comparison and acceptance; 5+10. Total: 35 hours of theory and analysis and 40 of practice.

Each 10-hour theory block comprises five 2-hour sessions; split the final 5 hours as 2+2+1. Place four 10-hour practicals in Sunday work intervals or divide them without increasing the total time. The numbers below denote module study hours. Overlapping module-boundary dates do not double the available time; retain holidays, rest and separate integration checkpoints from the main roadmap.

Fix the protocol before running algorithms

Save the sequence duration and boundaries, input sensors, frequencies, exact reference-trajectory source and map resolution. State whether the algorithm knows the initial position and orientation. Ground truth is only for error calculation; SLAM must not receive it through an accessible coordinate frame. Accidentally supplying ideal odometry invalidates the experiment.

Create bags/manifest, configs/lidar, configs/visual, trajectories, maps, evaluation and failures directories. For each run, save run_id, random-number seed, commit, noise parameters, data and trajectory-alignment method. Include matched timestamps, excluded intervals and the tracked-route fraction in evaluation, alongside aggregate metrics. Otherwise, a method that loses half the route can appear more accurate than a reliable baseline.

Experiment without searching every combination

Change one parameter class at a time: odometry noise, scan dropouts, initial offset or loop-closure threshold. Use three interpretable levels and retain one route. Tune on a separate sequence and evaluate the final result on a held-out one. You do not need to study all of Nav2: use only localisation and the map in M15. First verify visual SLAM on a short valid recording, then on one degraded-data episode; leave analysis time after installation.

Odometry and EKF: estimating motion with uncertainty

Essential concepts

Dead reckoning accumulates small displacements: for a planar body, $x_{\text{new}} = x + v \cdot \cos(\text{yaw}) \cdot dt$, $y_{\text{new}} = y + v \cdot \sin(\text{yaw}) \cdot dt$, $\text{yaw}_{\text{new}} = \text{yaw} + w \cdot dt$. Here v is in m/s, w in rad/s and dt in seconds. This discretisation suits a teaching calculation with small steps; for rapid turns, check the midpoint method or exact model integration. Bias is a constant offset, whereas noise is a varying deviation; they affect the trajectory differently.

In odom, the local trajectory should remain continuous even as drift accumulates. The map frame allows global map-based correction. Study the map $\rightarrow$ odom $\rightarrow$ base_link chain and the sensor's fixed pose relative to the body. One agreed source must publish each transform. A jump in the global estimate must not silently become a jump in the local controller's command.

Task 1. Body drift on a closed route

Generate a square trajectory with stops and saved true positions and orientations. Separately add velocity noise, a constant yaw-rate bias and slip on one turn. Integrate odometry and plot error against time and distance. Output: three trajectories and a parameter log. Test zero noise, $dt=0$ and timestamp discontinuity; no motion must mean no progress along the route.

Task 2. Fuse measurements and check covariance

Use state $[x,y,\text{yaw}]$, a motion model and periodic absolute position measurements in simulation. Write out prediction and correction steps, residual and covariance. Implement a compact EKF or adapt a verified example from earlier modules; compare with configured robot_localization on ROS data. Do not count the same information twice: position and velocity derived from the same encoder are correlated.

Output: the estimate, uncertainty ellipses and an innovation plot. Test missing absolute measurements, an anomalous jump and an artificially small R . For the leg, separately explain how an incorrect stationary-foot assumption produces false confidence. The task criterion is explainable error growth and filter responses, rather than a promise that EKF always improves the estimate. A poor model or wrong units can make it worse than simple odometry.

Maps and scan matching: what measurements provide

This week's topics

An occupancy grid stores the probability that a cell is occupied, not floor height. The inverse sensor model uses a ray: cells before a reliable return were observed free, cells at the return are probably occupied, and cells behind it remain unknown. Study log-odds $L = \log(p/(1-p))$, which is convenient for updates. Limit confidence accumulated from old observations. Define separate rules for missing measurements and maximum range.

2D scan matching estimates a transform between scans or between a scan and a map. ICP depends on correspondences and its initial estimate; a uniform long corridor poorly constrains motion along its length. AMCL represents a distribution of possible positions and orientations as particles on a known map: it localises rather than constructs a map. Compare a broad initial distribution across the map with a precisely specified initial pose.

Task 3. Build a map from known poses

Input: 200-500 simulated LaserScan messages and true poses, available only to this reference task. Transform laser points into the world, update free cells along rays and occupied cells at endpoints. Output: a grid, an occupied/free/unknown legend and the resolution setting. Check one ray and a wall by hand; do not mark the unseen side of a wall as free. Repeat with twice the cell size and compare wall thickness.

Task 4. LiDAR recording and localisation

Run `slam_toolbox` on the same route with ordinary odometry; save the pose graph and map. Then stop mapping and localise on the fixed map using AMCL or a supported localization mode. Do not run two `map→odom` publishers simultaneously. Save launch and config files, two trajectories, the map and a list of TF checks. Compare with your reference map from Task 3.

Test an incorrect initial pose, loss of 20% of scans and a laser rotation in the extrinsics. For scan matching, create a small pair with a known displacement and measure transform error, then use an ambiguous corridor pair. Separate mapping error from localisation error on an already known map in the report. Successful loop closure does not establish correct local geometry along every section.

Pose graphs: constraints, loop closure and false links

Sections to study

A graph vertex represents the robot's position and orientation, or pose; an edge represents measured relative motion or another constraint. Optimisation adjusts poses to reduce weighted residuals. Weights relate to uncertainty: using identical numbers for metres and radians without justification creates an arbitrary trade-off. Fixing one initial pose removes ambiguity in overall translation and rotation, but adds no measurements of the surroundings.

Loop closure recognises a previously visited place and adds a constraint. The optimiser reconciles constraints but does not verify correspondence truth. A robust loss reduces the influence of large residuals; it does not replace geometric verification and may fail to protect against a consistent false closure. Treat covariance as a local estimate within model assumptions, rather than the probability that the whole map is correct.

Task 5. A small graph you can understand

Create 20-30 planar poses around a square, noisy odometry edges and one correct loop-closure edge. Use GTSAM Pose2 or a small educational solver. Show initial and optimised trajectories, residuals and the method used to anchor the common coordinate frame. Save graph.json, noise settings and plots before and after optimisation. Test zero noise and a disabled prior; explain why absolute coordinates are undetermined without an anchor.

Task 6. False loop closure in a similar corridor

Add an edge between different but visually similar places. Compare ordinary squared error, a robust loss and geometric rejection of the incorrect link before optimisation. Measure ATE at the same poses and changes in residuals of correct edges. Save three results and the loop-acceptance rule. Show a case where the robust loss still allows map deformation; do not promise a universal threshold.

A minimal visual-odometry experiment

Bring calibration, RGB-D recordings and pair-registration checks from M14; in M15, assemble a short RGB-D odometry sequence. Build a graph of successive poses and add one revisit confirmed by image or cloud geometry. Use ground truth only for error evaluation. You may use an existing compatible visual SLAM system. If a loop closure is specified manually to test the optimiser, label it as a manual constraint: that experiment does not test automatic revisit recognition.

Evaluation: trajectory, map and tracking loss

What ATE and RPE measure

ATE compares estimated absolute poses with a reference after explicitly described alignment. For translation, report the RMSE of distances in metres in the usual formulation. RPE compares relative displacements between pairs of poses separated by a fixed interval, revealing local drift. Specify the interval in seconds, frames or metres and the timestamp-matching rule. Different intervals answer different questions and produce incomparable values.

For metric RGB-D and LiDAR, do not rescale trajectories during alignment except in a separate scale-error study. For a monocular baseline, state whether Sim(3) was used and which errors scale fitting conceals. Alongside aligned error, save the original trajectory, route coverage and the time tracking was lost. Do not evaluate only a successful short fragment while omitting the lost return leg.

Task 7. An evaluator you can trust

Create a reference trajectory, a copy with a known translation, a copy with slow drift and a time-shifted sequence. Run evo and independently calculate several pairs. Save commands, alignment settings, matched timestamps and an ATE/RPE table. After SE(3)/SE(2) alignment, overall translation and rotation should disappear while local drift remains. Specify the maximum timestamp mismatch and show the number of rejected poses.

Task 8. Map quality and robustness

In the simulated scene, compare reconstructed walls with known geometry: distances to reference lines, passage width and occupied/free-cell fractions in a predefined region. Do not count unknown as free. For each pipeline, report latency, tracking-recovery time and route coverage. Choose tolerances from map resolution and the clearance required by the future body; this is a teaching requirement, not a regulatory standard.

Offset sensor and odometry timestamps, then change the laser extrinsics; repeat with algorithm parameters unchanged. Explain why a synchronisation error can resemble a scan-matching error. Add a blurred section and loss of texture to the visual experiment. The overall conclusion must identify operating conditions for each method rather than declare one algorithm best under every condition.

Laboratory comparison of LiDAR and visual SLAM

Inputs and outputs

Inputs: recording manifest, calibration/frames and configs for three variants - odometry only, LiDAR SLAM and visual RGB-D SLAM. Outputs: trajectory with timestamp and pose fields, tracking status, a map or aligned keyframes, a loop-closure log and metrics. A separate error-evaluation programme receives the reference trajectory. For each variant, state available sensors, whether the starting pose is known and the source of metric scale.

Task 9. One question set for three methods

1. Select a valid sequence and a held-out difficult route. 2. Check TF and time bases without running SLAM. 3. Compute odometry alone. 4. Run the LiDAR variant and save the map and graph. 5. Run the visual method with fixed parameters. 6. Evaluate matching available intervals and full coverage separately. 7. Repeat with one fault. 8. Assemble a report explaining the causes of differences.

If the camera and laser were recorded in the same simulator, compare identical reference-trajectory sections. If RGB-D comes from TUM, create two tables: metrics within each dataset and a qualitative comparison of failures. Claims such as "LiDAR is N times more accurate" are invalid for different routes. Tabulate comparison conditions so scene differences remain explicit.

Task 10. Robot response to lost localisation

Create a component that receives pose with status, timestamp and uncertainty. For stale data or lost tracking, it must output `localization_unavailable` and reject new global goals in simulation. Do not restart the running local stability loop. Recovery requires a new valid estimate, reconciliation of map/odom and an explicit return transition. Save an event log and a missing-data test.

Deliverable package

The README must describe recording processing and error evaluation; the manifest documents data provenance; configs contains every parameter. Include ATE/RPE with definitions, map quality, latency, route coverage and failure scenarios in the report. Add a TF-tree snapshot and example input/output messages. For the future hexapod, state the applicability boundary: global localisation helps follow a route but does not validate leg stability or contact reliability.

Eight failure and sensitivity scenarios

Faults to introduce

S1 - No absolute observations: disable EKF corrections and loop closures along part of the route. Expect growing drift and increased uncertainty if the model accounts for it. Compare with zero noise and check for false confidence despite nonzero error. S2 - Slip: add odometry error on one turn. Save the fault time, local RPE and correction after observing the surroundings.

S3 - Corridor ambiguity: walls repeat and transverse structure is sparse. Measure errors along and across the corridor separately; a small ICP residual does not yet establish correct displacement. S4 - False closure: connect similar places with an incorrect edge. Evaluate map deformation, ATE and changes in correct-edge residuals with and without a robust loss.

S5 - Kidnapped robot: in simulation, instantaneously move the body elsewhere on the known map without a corresponding odometry change. Test AMCL with local and broad initial distributions and measure recovery time under the chosen criterion. Manual reinitialisation must not be called automatic relocalisation. S6 - Data loss: remove scans or images for 1 and 3 seconds. The consumer must detect estimate age and unavailability.

S7 - Time and extrinsics: shift sensor timestamps by one period, then separately rotate the sensor by a known angle. Compare errors on straight sections and turns. S8 - Image degradation: blur, a smooth wall or a covered camera. Measure the fraction of tracked images and test recovery; errors on the surviving images alone are insufficient. Do not remove failed sections before preparing the final table.

Choosing levels and conclusions

Fault levels are illustrative parameters chosen to suit the measurement frequency and scene scale; they are not allowable limits for a physical robot. Change one cause per experiment and preserve the seed and original bag. Evaluate sensitivity at three levels; state whether improvement repeats or occurs only on one route.

Each scenario needs an expected-result file, actual trace, numerical criterion or manual-check rule, and a decision: handled, detected without recovery, or undetected. The last outcome is acceptable as an established limitation of this introductory SLAM work, but a consumer must not present an unreliable pose estimate as current and accurate.

Sessions 1-40: motion, the filter and the map

Hours 1-10: theory in 2-hour sessions

1-2: wheel, leg, visual and inertial odometry models, with one failure-prone assumption each. 3-4: planar-motion integration and bias; calculate a turn and return by hand. 5-6: map/odom/base_link, transform directions and TF sources. 7-8: EKF prediction and correction, Q/R and measurement correlation. 9-10: ground truth, error units and the Task 1 protocol. Record these principles as implementation assumptions.

Hours 11-20: practical 1

11-12: square-route generator and an error-free check. 13-14: Task 1 - noise, bias and local slip. 15-16: Task 2 - EKF, covariance and innovation plots. 17-18: process a ROS recording, compare with the chosen filter and check TF. 19-20: missing measurements, excessively small R and an error report. Understand the source of drift before connecting SLAM.

Hours 21-30: theory in 2-hour sessions

21-22: occupancy, free/occupied/unknown and log-odds. 23-24: inverse laser-ray model, resolution and invalid range. 25-26: scan matching, ICP, local minima and the corridor. 27-28: particles, measurement likelihood and AMCL resampling through an understandable example. 29-30: slam_toolbox modes, the difference between a map and a graph, and state saving. Select the manual parameters needed for your scenario.

Hours 31-40: practical 2

31-32: Task 3, a map from known poses and a manual ray check. 33-34: a lower-resolution map and the unknown/free distinction. 35-36: Task 4, process the LiDAR SLAM recording and save the graph and map. 37-38: localisation on a fixed map, initial offset and TF transform sources. 39-40: corridor, scan loss and a comparison report against the reference map.

By this stage, there should be three clearly understood files: a reference map for evaluation, a reconstructed map and pose estimates. Identify their roles in filenames and the manifest. If map processing continues after playback stops, allow the queue to finish within the practical and record processing lag; do not compare one method's partial result with another's completed result.

Sessions 41-75: graphs, comparison and acceptance

Hours 41-50: theory in 2-hour sessions

41-42: graph vertices and edges, residuals, noise and units. 43-44: nonlinear least squares, linearisation and initial estimates; one worked manual example is enough. 45-46: the prior, common-frame ambiguity and covariance meaning. 47-48: loop closure, geometric verification and robust losses. 49-50: visual-method structure; choose an existing baseline or small RGB-D graph before the practical.

Hours 51-60: practical 3

51-52: Task 5, a graph of 20-30 poses, noise and correct closure. 53-54: Task 6, a false link and three handling methods. 55-56: visual odometry on a short M14 sequence. 57-58: a keyframe map and verified closure, or running an existing visual SLAM system. 59-60: save trajectories, tracking status and a reproducible launch. If you used a manually specified closure edge, label it in the experiment name.

Hours 61-65: theory 2+2+1

61-62: ATE/RPE, intervals, alignment and scale; calculate three pairs by hand. 63-64: route coverage, timestamp matching, map quality and limits of cross-dataset comparison. 65: prepare the final fault protocol and localisation-availability decision. Justify numerical thresholds in terms of scene scale rather than copying them from another sensor.

Hours 66-75: practical 4

66-67: Task 7 - check the error-evaluation programme with a known transform and drift. 68-69: Task 8 and the three-method comparison from Task 9 on held-out data. 70-71: S1-S4 with fixed parameters and a sensitivity table. 72-73: S5-S8 and Task 10, including status recovery. 74-75: reproduce the final package and hand the map and status to M16. All intervals together total 75 hours.

If time is limited

First reduce the number of long recordings and parameters, then omit extra optimisers and detailed 3D visualisation. Retain processing of one LiDAR recording, one visual baseline, a small understandable pose graph, ATE/RPE, route coverage and consumer rejection of stale estimates. Do not spend the rest of the month porting SLAM to Rust. State unresolved integration issues: introductory coverage does not require results matching leading academic benchmarks.

Completion criteria and data for the planner

Acceptance checks

Quantitatively compare at least uncorrected odometry and SLAM on one sequence. Add a separate visual method with a map or keyframes and failure analysis. For ATE/RPE, state definitions, units, interval, alignment method, matched-sample fraction and route coverage. Tabulate available sensors, reference data and condition differences. Incomparable scenes cannot establish sensor superiority.

Find and explain examples of drift, degenerate scan matching, false closure, timing error and sensor loss. Save settings for the valid run and modified versions. The map must contain resolution, origin and frame_id, with unknown/free/occupied distinguished. Recalculating errors must not require manual trajectory edits. Any component using pose must respond to stale data and unconfirmed recovery.

Self-check

1. How does map differ from odom, and who publishes their transform? 2. Why does leg odometry fail during slip? 3. What happens with an incorrect dt? 4. Why can an excessively small R worsen EKF performance? 5. How does an unknown cell differ from a free one? 6. Why does scan matching constrain some motion directions better than others? 7. How does a false loop edge change the whole trajectory? 8. What does a prior fix, and what does it not measure? 9. How can ATE improve after failed frames are removed, and why does this distort quality evaluation? 10. Why calculate RPE at several intervals? 11. When is scale correction permissible? 12. Why does good SLAM not establish hexapod stability?

Handover to M16

Pass on the map, coordinate-frame chain, start and goal in one frame, an example localisation status and a data-age limit. Add two maps: one valid and one with a known error or a passage through an unknown region. The planner must reject a task when the pose estimate is unavailable. A map alone does not establish whether an area is suitable as a foothold.

Demonstrate a normal circuit, a section with odometry error, a false closure and missing data. Show numbers, then explain one failure cause without saying merely "poorly tuned". Frame the final conclusion in terms of operating conditions: "in this scene, with these sensors and parameters". Global navigation broadens the topics covered; local body and leg control can develop using other state-estimation sources.

Assigned reading: a focused route through the sections

The literature is mainly in English; the programme and definitions are also in English. Read the selected sections before the relevant task, rather than entire books. Core reading is included in the existing 35 analysis hours; consult references as needed.

Core reading

S. Thrun, W. Burgard, D. Fox, Probabilistic Robotics (2005), accompanying teaching materials. Authors' slides: kalman.ppt - Bayesian filtering and EKF; mapping-occupancy.ppt - occupancy updates; slam.ppt - joint map and trajectory estimation. Before Tasks 1-4, record states, observations and assumptions. The MIT Press book provides detailed reference material on these topics.

F. Dellaert, Factor Graphs and GTSAM: A Hands-on Introduction, teaching notes. Author's GTSAM notes. §1 Factor Graphs, §3 Robot Localization, §4 PoseSLAM; for the prior example, §2.2-2.4. In Tasks 5-6, build a small Pose2 graph with a prior and relative constraints, then add one false closure.

Reference reading

R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. (2022). Author's book. Ch. 11 Structure from motion and SLAM: selected material on pose estimation, bundle adjustment and SLAM. For visual experiments in Tasks 5-7, connect correspondences, camera motion, error accumulation and map correction.

Documentation for your experiment

ROS REP-105 - odom, map, Relationship between Frames. In Tasks 1-2, explicitly assign each TF source and the location of global correction.

slam_toolbox - Parameters, Localization and serialization. For Task 4, learn to distinguish a saved map image from a graph that can support continued operation.

evo: Metrics - Absolute pose error, Relative pose error, pose relation and alignment. Before Task 7, choose units, time association and permissible trajectory alignment.

Minimum outcome from reading

Explain using your own route: where odom drifts, how map corrects global error, what connects two poses in the graph, and why a plausible-looking map can conceal a false closure. Downloading large amounts of KITTI and writing a second full SLAM system from scratch are not required.

Terms and definitions

Localisation. Estimating the robot's position and orientation relative to a known coordinate frame or map using available measurements.

Odometry. Estimating relative motion from successive measurements; accumulated error usually grows without external correction.

SLAM. Joint estimation of trajectory and map, linking motion uncertainty to uncertainty in observed objects.

Prior. Information about the state before incorporating the current measurement; it may come from a motion model or initial condition.

Likelihood. A function describing the probability of an observation for a given state and sensor model; by itself, it is not a state distribution.

Posterior. The state distribution after combining prior information and a measurement under the adopted probabilistic model.

EKF. A filter that approximates nonlinear models through local linearisation and represents its estimate by a mean and covariance.

Innovation. The difference between a measurement and its prediction, interpreted with the uncertainty of both prediction and sensor.

Covariance. A matrix of variances and joint deviations of estimate components; its units depend on products of the corresponding component units.

odom frame. A local frame with a continuous trajectory that can gradually drift relative to the surrounding world.

map frame. A global map frame that allows correction jumps when accumulated drift in the local estimate is detected.

Occupancy grid. A map of spatial-cell occupancy probabilities; an unobserved cell must differ from one confirmed free.

Log-odds. The logarithm of the probability ratio between an event and its absence; convenient for occupancy updates.

Scan matching. Estimation of a transform from spatial measurements; it may be ambiguous with sparse or repetitive geometry.

Pose graph. A representation of a trajectory as pose vertices and constraints between them, obtained from motion or repeated observations.

Factor. A local likelihood or cost function relating a subset of unknown variables to a specific measurement or condition.

Loop closure. A constraint between the current and a previously visited place, capable of correcting drift if the correspondence is recognised correctly.

Gauge freedom. Ambiguity in the overall pose under relative constraints; removed by anchoring or an absolute observation.

ATE. Error of estimated poses relative to reference poses after explicitly specified alignment; it depends on time association and the selected component.

RPE. Relative-motion error between pose pairs over a declared temporal or spatial interval; characterises local drift.

Equipment and setup for this month

Use existing equipment

MacBook Pro M4, Ubuntu 24.04 ARM64 in UTM, Jazzy and Harmonic, the M14 RGB-D camera if available, and saved sequences. Simulated LaserScan with ground truth is sufficient for the required 2D experiment; purchasing LiDAR is not required this month.

Purchase for a physical LiDAR experiment

If desired, prepare 1 two-dimensional LiDAR with a ROS 2 Jazzy driver for your computer and documented range, angles, frequency and timestamps. Before ordering, check its ARM64 SDK, power and complete USB/serial/Ethernet communication path; select one supported unit. You need a compatible data cable and 1 mount. The purchase extends physical data collection but is optional for the teaching comparison.

Install now

Add slam_toolbox from a Jazzy branch or package and robot_localization for the EKF task to the existing Ubuntu workspace. Use `ros-jazzy-slam-toolbox`; configure `robot_localization` only for fields actually supplied. For the known-map experiment, install the Jazzy nav2_amcl component now if absent; the full Nav2 workflow is deployed in M16.

For trajectory evaluation, add `evo` in a separate Python environment and record its version. You may use GTSAM for the small pose graph: install an ARM64 package or build a fixed release if you choose this option. Alternatively, write a small least-squares graph using the existing NumPy/SciPy. For RGB-D odometry, use Open3D from M14.

Check the installation

Replay a short bag containing LaserScan, odom and IMU. Check the single map → odom → base_link chain, build a map and save the pose graph. Reload the graph and check continued operation. For EKF, provide a known displacement with correct covariances; for AMCL, provide a known map and initial position and orientation.

Check metrics on a trajectory identical to ground truth, then on a known displacement; explicitly specify alignment and scale. A small three-pose graph must solve with a prior and detect an added false link. Check new peripheral USB connections separately; successful recording processing does not establish actual synchronisation.

Preparation and initial functional checks fit within the existing 75 hours in place of repeated installations; optional LiDAR delivery is outside sessions. Do not repurchase the camera, IMU, MCU, motor or computer.

Motion fundamentals: support and odometry

Required: how a leg measures body motion

When the body moves above a stationary foot, joint angles change even though world-frame contact velocity is zero. Encoders measure relative leg geometry; body velocity can be derived by adding leg kinematics and IMU angular velocity. Before deriving the equations, check foot FK and the cross product: rotation produces a linear velocity $\omega \times r$ that depends on the lever arm.

From position to a velocity constraint

W is the world, B the body and F a fixed material point on the foot. $p_{WF} = p_{WB} + R_{WB} p_{BF}(q)$; q contains only leg joint angles in radians. $J_p = \partial p_{BF} / \partial q$ is the $3 \times n$ position Jacobian in B , so relative foot velocity is $J_p \dot{q}$. This is not the foot's complete world-frame velocity.

Let v_{WB} be the linear velocity of the B origin expressed in W , and ω_B the body angular velocity expressed in B . Differentiating gives $v_{WF} = v_{WB} + R_{WB}(\omega_B \times p_{BF} + J_p \dot{q})$. All linear velocities are in m/s; ω_B is in rad/s. Only a stationary, non-slipping support gives $v_{WF} = 0$, hence $v_{WB} = -R_{WB}(\omega_B \times p_{BF} + J_p \dot{q})$. Contact alone does not justify setting this velocity to zero.

Example with body rotation

At one instant, $R_{WB} = I$, $p_{BF} = (0.2; 0; -0.3)$ m, $\omega_B = (0; 0; 0.5)$ rad/s and $J_p \dot{q} = (-0.04; 0; 0)$ m/s. The rotational contribution is $\omega_B \times p_{BF} = (0; 0.10; 0)$ m/s. This gives $v_{WB} = (0.04; -0.10; 0)$ m/s. Omitting rotation loses the lateral motion of 0.10 m/s, even though FK and encoders are correct.

If the same foot actually slips at $v_{WF} = (0.02; 0; 0)$ m/s, the true $v_{WB} = (0.06; -0.10; 0)$ m/s. The incorrect zero-velocity assumption gives an x-error of -0.02 m/s. Several reliable supports allow residual comparisons; a shared geometry error or simultaneous slip can bias estimates from all legs.

Exercises and evidence of understanding

1. Add a short synthetic stance episode to Task 1: calculate world-frame velocity in two ways, by differentiating p_{WF} and by summing the three terms. Save a table and a residual plot; halving the finite-difference step should reduce error until rounding error dominates. Ground truth is available only to metric calculation, not to the state estimator.

2. In Task 2, repeat the example with slipping and loss of contact. Save the innovation and support-admissibility flag; disabling an unreliable constraint must be visible. Explain why integrating velocities produces drift and does not by itself determine absolute position and orientation in map.

Time accounting: 1 hour from 1-2 for the leg-odometry model; 1 hour each from 13-14 and 15-16 for examples within Tasks 1-2. The total remains 75 hours; do not add physical walking or a new filter beyond these tasks.

Assigned reading: Lynch/Park, [Modern Robotics §3.2.2 Angular Velocities](#) and [§5.1.2 Body Jacobian](#). Distinguish the book's six-dimensional spatial velocity (twist) from the point-position Jacobian used here.